

NiceGUI 中 app.middleware 深度解析

在 NiceGUI 框架中，`app.middleware` 是基于底层 FastAPI/Starlette 中间件机制实现的**请求处理扩展工具**，用于在 HTTP 请求的生命周期中插入自定义逻辑，实现请求拦截、响应修改、权限验证、日志记录、跨域处理等核心功能。中间件作为请求处理的“管道”，可以在请求到达页面处理函数前、响应返回客户端前执行自定义代码，是扩展 NiceGUI 应用底层能力的关键入口。本文将从**核心原理**、**注册方式**、**内置中间件**、**自定义中间件**、**典型应用场景**、**注意事项**六个维度，全方位解析 NiceGUI 的 `app.middleware` 机制。

一、app.middleware 的核心原理

NiceGUI 基于 FastAPI 构建，而 FastAPI 又继承了 Starlette 的中间件体系，因此 `app.middleware` 的底层原理与 Starlette 完全一致，核心遵循**请求 - 响应生命周期**的拦截机制。

1.1 中间件的执行流程

一个 HTTP 请求从客户端发送到 NiceGUI 应用，再到响应返回的完整生命周期中，中间件的执行顺序如下：

- 请求到达服务器**：客户端的 HTTP 请求首先被 NiceGUI 底层的 ASGI 服务器（如 Uvicorn）接收；
- 中间件链处理请求**：请求按**注册顺序**依次经过所有已注册的中间件，中间件可修改请求头、验证权限、记录日志等；
- 页面处理函数执行**：经过中间件处理后的请求，到达 `ui.page` 装饰的页面处理函数，生成响应数据；
- 中间件链处理响应**：页面处理函数生成的响应，按**注册逆序**再次经过所有中间件，中间件可修改响应头、格式化响应数据、添加缓存策略等；
- 响应返回客户端**：最终的响应被发送回客户端，完成一次请求 - 响应循环。

简单来说，中间件是请求的“前置过滤器”**和**响应的“后置处理器”，形成一个双向的处理链。

1.2 中间件的类型

在 NiceGUI 中，通过 `app.middleware` 可注册两种核心类型的中间件，对应不同的处理粒度：

中间件类型	注册方式	处理对象	适用场景
HTTP 中间件	<code>app.middleware('http')</code>	所有 HTTP 请求 / 响应（包括页面请求、静态资源请求、API 请求）	全局请求拦截、跨域处理、日志记录、权限验证
WebSocket 中间件	<code>app.middleware('websocket')</code>	WebSocket 连接请求（NiceGUI 前后端实时通信的核心）	WebSocket 连接验证、心跳检测、消息拦截

NiceGUI 依赖 WebSocket 实现前后端的实时交互（如组件事件回调、数据同步），因此除了 HTTP 中间件，WebSocket 中间件也具备重要的扩展价值。

二、app.middleware 的注册方式

NiceGUI 提供了**装饰器**和**手动注册**两种中间件注册方式，其中装饰器方式是最常用的，手动注册则适用于复杂的中间件类。

2.1 装饰器注册（推荐）

通过 `@app.middleware('类型')` 装饰器可快速注册中间件函数，函数需接收 `request`（请求对象）和 `call_next`（下一个中间件 / 处理函数的调用方法）两个参数，返回值为响应对象。

2.1.1 HTTP 中间件注册示例

```
from nicegui import ui, app
from starlette.responses import Response

# 注册 HTTP 中间件
@app.middleware('http')
async def custom_http_middleware(request, call_next):
    # 【请求处理阶段】：请求到达页面处理函数前执行的逻辑
    print(f'收到请求: {request.method} {request.url}')
    print(f'请求头: {dict(request.headers)}')

    # 调用下一个中间件/页面处理函数，获取响应
    response: Response = await call_next(request)

    # 【响应处理阶段】：响应返回客户端前执行的逻辑
    response.headers['X-Custom-Header'] = 'NiceGUI-Middleware' # 添加自定义响应头
    print(f'返回响应: {response.status_code}')

    return response

@ui.page('/')
def index():
    ui.label('中间件测试页面').classes('text-3xl')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

2.1.2 WebSocket 中间件注册示例

```
from nicegui import ui, app

# 注册 websocket 中间件
@app.middleware('websocket')
async def custom_websocket_middleware(websocket, call_next):
    # 【连接建立前】：验证 websocket 连接请求
    print(f'WebSocket 连接请求: {websocket.url}')
    print(f'客户端 IP: {websocket.client}')

    # 调用下一个中间件/建立连接
    await call_next(websocket)

    # 【连接关闭后】：执行清理逻辑
    print('WebSocket 连接已关闭')

@ui.page('/')
def index():
```

```
ui.button('点击触发 websocket 通信', on_click=lambda: ui.notify('测试'))

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

2.2 手动注册中间件类

对于复杂的中间件逻辑，可通过定义**中间件类**（继承自 Starlette 的 `BaseHTTPMiddleware` 或 `WebSocketMiddleware`），再通过 `app.add_middleware()` 手动注册，适用于需要封装状态或复杂逻辑的场景。

```
from nicegui import ui, app
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.responses import Response

# 定义自定义 HTTP 中间件类
class LoggingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next) -> Response:
        # 请求处理逻辑
        print(f'请求路径: {request.url.path}')
        # 调用下一个处理环节
        response = await call_next(request)
        # 响应处理逻辑
        response.headers['X-Logging'] = 'Enabled'
        return response

# 手动注册中间件
app.add_middleware(LoggingMiddleware)

@ui.page('/')
def index():
    ui.label('手动注册中间件测试').classes('text-3xl')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

三、NiceGUI 常用的内置中间件

FastAPI/Starlette 提供了一系列开箱即用的中间件，可通过 `app.add_middleware()` 直接注册到 NiceGUI 应用中，覆盖跨域、缓存、会话、压缩等常见需求，无需手动实现。

3.1 跨域中间件 (CORSMiddleware)

解决前端跨域请求的核心中间件，适用于 NiceGUI 应用作为后端提供 API 服务，被其他域名的前端页面调用的场景。

```
from nicegui import ui, app
from starlette.middleware.cors import CORSMiddleware

# 注册跨域中间件
app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'], # 允许所有域名访问（生产环境需指定具体域名）
```

```

    allow_credentials=True,
    allow_methods=['*'], # 允许所有 HTTP 方法
    allow_headers=['*'], # 允许所有请求头
)

@ui.page('/api/data')
def api_data():
    return {'message': 'Hello NiceGUI'} # 返回 JSON 数据

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

3.2 压缩中间件 (GZipMiddleware)

对响应数据进行 GZip 压缩，减少网络传输体积，提升加载速度。

```

from nicegui import ui, app
from starlette.middleware.gzip import GZipMiddleware

# 注册 GZip 压缩中间件，压缩阈值为 1024 字节
app.add_middleware(GZipMiddleware, minimum_size=1024)

@ui.page('/')
def index():
    # 大体积文本，触发压缩
    ui.label('Lorem ipsum ' * 1000).classes('text-lg')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

3.3 会话中间件 (SessionMiddleware)

基于加密 Cookie 实现的会话管理中间件，补充 NiceGUI 内置 `ui.session` 的能力，适用于需要持久化会话数据的场景。

```

from nicegui import ui, app
from starlette.middleware.sessions import SessionMiddleware
import secrets

# 生成加密密钥（生产环境需保存在环境变量中）
SECRET_KEY = secrets.token_hex(16)

# 注册会话中间件
app.add_middleware(SessionMiddleware, secret_key=SECRET_KEY)

@app.middleware('http')
async def session_middleware(request, call_next):
    # 从请求中获取会话数据
    request.session.setdefault('visit_count', 0)
    request.session['visit_count'] += 1
    response = await call_next(request)

```

```

# 在响应中添加会话信息
response.headers['X-Visit-Count'] = str(request.session['visit_count'])
return response

@ui.page('/')
def index():
    ui.label('会话访问次数统计').classes('text-3x1')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

3.4 信任代理中间件 (TrustedHostMiddleware/ForwardedForMiddleware)

适用于 NiceGUI 应用部署在反向代理（如 Nginx、Traefik）后的场景，用于获取客户端真实 IP、验证请求主机名。

```

from nicegui import ui, app
from starlette.middleware.trustedhost import TrustedHostMiddleware
from starlette.middleware.forwarded import ForwardedForMiddleware

# 验证请求主机名，仅允许指定域名访问
app.add_middleware(TrustedHostMiddleware, allowed_hosts=['example.com', 'www.example.com'])

# 获取客户端真实 IP（反向代理后）
app.add_middleware(ForwardedForMiddleware)

@app.middleware('http')
async def real_ip_middleware(request, call_next):
    print(f'客户端真实 IP: {request.client.host}')
    response = await call_next(request)
    return response

@ui.page('/')
def index():
    ui.label('反向代理中间件测试').classes('text-3x1')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

四、自定义中间件的典型应用场景

`app.middleware` 的核心价值在于通过自定义逻辑扩展 NiceGUI 应用的底层能力，以下是开发中最常见的自定义中间件应用场景，覆盖权限验证、日志记录、请求限流等核心需求。

4.1 全局权限验证中间件

实现对所有页面请求的权限拦截，仅允许已登录用户访问受保护的路径，是前后端分离或多页面应用的核心需求。

```

from nicegui import ui, app
from starlette.responses import RedirectResponse

```

```

# 公开路径（无需登录）
PUBLIC_PATHS = ['/', '/login']

@app.middleware('http')
async def auth_middleware(request, call_next):
    # 跳过公开路径的验证
    if request.url.path in PUBLIC_PATHS:
        return await call_next(request)

    # 从会话中获取登录状态（结合 NiceGUI 的 ui.session）
    session_id = request.cookies.get('nicegui-session')
    if not session_id or not app.storage.session.get(session_id, {}).get('is_login'):
        # 未登录则重定向到登录页
        return RedirectResponse(url='/login')

    # 已登录则继续处理请求
    response = await call_next(request)
    return response

@ui.page('/')
def index():
    ui.button('前往仪表盘', on_click=lambda: ui.navigate.to('/dashboard'))

@ui.page('/login')
def login():
    def do_login():
        # 标记登录状态到会话
        app.storage.session[ui.session.id]['is_login'] = True
        ui.navigate.to('/dashboard')

    ui.button('模拟登录', on_click=do_login)

@ui.page('/dashboard')
def dashboard():
    ui.label('仪表盘（需登录）').classes('text-3xl')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

4.2 全局请求日志中间件

记录所有 HTTP 请求的详细信息（方法、路径、状态码、耗时等），用于应用的监控和故障排查，是生产环境的必备功能。

```

from nicegui import ui, app
import time
import logging

# 配置日志
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(message)s')
logger = logging.getLogger(__name__)

```

```

@app.middleware('http')
async def log_middleware(request, call_next):
    # 记录请求开始时间
    start_time = time.time()
    # 处理请求
    response = await call_next(request)
    # 计算请求耗时
    process_time = time.time() - start_time
    # 记录日志
    logger.info(
        f'Method: {request.method}, Path: {request.url.path}, '
        f'Status: {response.status_code}, Time: {process_time:.2f}s'
    )
    return response

@ui.page('/')
def index():
    ui.label('日志中间件测试').classes('text-3xl')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

4.3 请求限流中间件

限制单个客户端的请求频率，防止恶意请求或高频访问导致服务器过载，适用于公开的 NiceGUI 应用。

```

from nicegui import ui, app
from starlette.responses import PlainTextResponse
import time
from collections import defaultdict

# 存储客户端的请求记录: {client_ip: [请求时间戳列表]}
request_records = defaultdict(list)
# 限流配置: 10 秒内最多 5 次请求
RATE_LIMIT = 5
TIME_WINDOW = 10

@app.middleware('http')
async def rate_limit_middleware(request, call_next):
    # 获取客户端 IP
    client_ip = request.client.host
    current_time = time.time()

    # 清理超时的请求记录
    request_records[client_ip] = [t for t in request_records[client_ip] if current_time - t
    < TIME_WINDOW]

    # 检查请求频率
    if len(request_records[client_ip]) >= RATE_LIMIT:
        return PlainTextResponse('请求过于频繁，请稍后再试', status_code=429)

    # 记录当前请求时间

```

```

request_records[client_ip].append(current_time)
response = await call_next(request)
return response

@ui.page('/')
def index():
    ui.label('限流中间件测试').classes('text-3x1')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

4.4 WebSocket 连接验证中间件

对 NiceGUI 的 WebSocket 连接进行权限验证，仅允许已授权的客户端建立实时通信，防止未授权的实时交互。

```

from nicegui import ui, app
from starlette.websockets import WebSocketDisconnect

@app.middleware('websocket')
async def websocket_auth_middleware(websocket, call_next):
    # 从查询参数中获取授权令牌
    token = websocket.query_params.get('token')
    # 验证令牌（生产环境需使用加密令牌）
    if token != 'valid_token':
        # 拒绝连接
        await websocket.close(code=1008) # 1008 = 策略违反
        return

    # 验证通过，建立连接
    try:
        await call_next(websocket)
    except WebSocketDisconnect:
        pass

@ui.page('/')
def index():
    # 带令牌的 WebSocket 连接（实际开发中通过前端传递）
    ui.button('测试 WebSocket 连接', on_click=lambda: ui.notify('已连接'))

if __name__ in {'__main__', '__mp_main__'}:
    # 启动时添加令牌参数（仅示例，实际通过前端传递）
    ui.run(params={'token': 'valid_token'})

```

五、使用 app.middleware 的注意事项

中间件作为请求处理的核心扩展，使用不当可能导致应用性能下降、请求阻塞或逻辑错误，需注意以下关键问题：

5.1 中间件的执行顺序

- **注册顺序决定请求处理顺序**：HTTP 中间件在请求阶段按**注册顺序**执行，响应阶段按**注册逆序**执行；
- **避免循环依赖**：中间件之间应保持独立，避免在中间件中调用依赖其他中间件的逻辑；
- **精简中间件链**：过多的中间件会增加请求处理的耗时，仅保留必要的中间件。

5.2 异步与同步的兼容性

NiceGUI 基于 ASGI 实现，中间件函数**必须使用异步**（`async/await`）编写，若需要执行同步阻塞操作（如数据库查询），需通过 `asyncio.to_thread` 封装，避免阻塞事件循环：

```
import asyncio
import time

def blocking_operation():
    """同步阻塞操作"""
    time.sleep(1)
    return 'done'

@app.middleware('http')
async def sync_middleware(request, call_next):
    # 将同步操作封装到线程中
    result = await asyncio.to_thread(blocking_operation)
    print(f'阻塞操作结果: {result}')
    response = await call_next(request)
    return response
```

5.3 避免修改核心请求 / 响应对象

- 不要随意修改 `request` 对象的核心属性（如 `method`、`url`），可能导致页面处理函数执行异常；
- 修改响应对象时，需确保响应头、状态码符合 HTTP 规范，避免客户端解析失败。

5.4 生产环境的安全性

- 权限验证中间件中，避免使用明文传递令牌 / 密码，应使用 HTTPS + 加密令牌（如 JWT）；
- 日志中间件中，避免记录敏感信息（如用户密码、请求头中的认证信息）；
- 跨域中间件中，生产环境不要设置 `allow_origins=['*']`，应指定具体的可信域名。

5.5 与 NiceGUI 内置功能的协同

- 中间件中访问 `ui.session` 时，需通过 `app.storage.session` 结合会话 ID 实现，避免直接在中间件中使用 `ui.session`（上下文不匹配）；
- 中间件的重定向逻辑（如 `RedirectResponse`）需与 `ui.navigate` 协同，确保前端路由的一致性。

六、总结

`app.middleware` 是 NiceGUI 应用**底层能力扩展的核心入口**，基于 FastAPI/Starlette 的中间件机制，实现了对 HTTP 请求和 WebSocket 连接的全生命周期拦截与处理。通过注册内置中间件，可快速实现跨域、压缩、会话管理等常见需求；通过自定义中间件，可封装权限验证、日志记录、请求限流等业务逻辑，显著提升应用的安全性、可监控性和扩展性。

开发中的最佳实践总结：

- 按需注册**：仅保留必要的中间件，避免性能损耗；
- 异步优先**：所有中间件函数使用 `async/await` 编写，同步操作通过线程封装；
- 安全第一**：生产环境中严格验证请求、加密敏感数据、限制跨域访问；
- 日志规范**：通过中间件记录关键请求信息，便于故障排查和应用监控；
- 协同开发**：中间件逻辑与 NiceGUI 内置的 `ui.session`、`ui.navigate` 等功能协同，确保整体流程的一致性。

通过合理使用 `app.middleware`，可将 NiceGUI 从简单的前端界面框架扩展为具备企业级特性的全栈应用，满足生产环境的复杂需求。

NiceGUI 中 `app.middleware` 的两种类型详解

NiceGUI 基于 FastAPI 构建，其 `app.middleware` 本质上复用了 FastAPI/Starlette 的中间件体系，核心分为**HTTP 中间件**和**WebSocket 中间件**两类，分别处理 HTTP 请求和 WebSocket 连接生命周期，以下是详细拆解：

一、核心概念：中间件的作用

中间件是介于客户端请求与 NiceGUI 应用处理逻辑之间的“拦截层”，可在请求 / 连接的**前置阶段**（如验证、日志）、**后置阶段**（如响应处理、清理）执行自定义逻辑，且支持链式调用。

NiceGUI 的 `app` 实例（`from nicegui import app`）直接继承了 Starlette 的 `Middleware` 注册能力，两类中间件的注册和触发逻辑因协议特性差异显著。

二、HTTP 中间件

1. 适用场景

处理所有 HTTP 类请求：页面访问（GET）、API 调用（POST/PUT/DELETE）、静态资源加载（如 CSS/JS）等，覆盖 NiceGUI 中除 WebSocket 外的所有请求类型。

2. 注册方式

通过 `app.add_middleware()` 注册，核心依赖 Starlette 的 `HTTPMiddleware`，需定义一个“中间件函数”接收 `request`（请求对象）和 `call_next`（执行下一个中间件 / 处理逻辑的回调）。

3. 核心参数与生命周期

- 触发时机**：客户端发起 HTTP 请求后，NiceGUI 路由处理前；响应返回客户端前。
- 核心逻辑流程**：
 - 接收客户端 HTTP 请求 → 执行前置逻辑（如鉴权、日志）；
 - 调用 `response = await call_next(request)` 传递请求到下一层；

3. 执行后置逻辑（如修改响应头、统计耗时）；
4. 返回 `response` 给客户端。

4. 示例代码

```
from nicegui import app, ui
from starlette.middleware.base import HTTPMiddleware
from starlette.requests import Request
from starlette.responses import Response
import time

# 定义HTTP中间件：记录请求耗时+鉴权
async def custom_http_middleware(request: Request, call_next) -> Response:
    # 前置逻辑：记录开始时间+验证token
    start_time = time.time()
    token = request.headers.get("Authorization")
    if token != "valid_token" and request.url.path != "/login":
        return Response("Unauthorized", status_code=401)

    # 传递请求到下一层（NiceGUI路由/处理逻辑）
    response = await call_next(request)

    # 后置逻辑：记录耗时并添加响应头
    process_time = time.time() - start_time
    response.headers["X-Process-Time"] = str(process_time)
    return response

# 注册HTTP中间件
app.add_middleware(HTTPMiddleware, dispatch=custom_http_middleware)

# 测试页面
@ui.page("/")
def index():
    ui.label("Hello NiceGUI!")

@ui.page("/login")
def login():
    ui.label("Login Page (No Auth)")

ui.run()
```

5. 关键特性

- 同步 / 异步兼容：中间件函数可定义为 `async`（推荐）或普通函数；
- 全 HTTP 方法覆盖：GET/POST/PUT/DELETE 等都会触发；
- 响应可修改：支持修改响应头、状态码甚至响应体。


```
        except:
            print(f"Received raw data: {data['text']}")
        return data

# 重写send方法: 拦截发往客户端的消息
original_send = websocket.send
async def wrapped_send(data):
    if data["type"] == "websocket.send":
        try:
            msg = json.loads(data["text"])
            print(f"Sending to client: {msg}")
            # 示例: 修改消息 (给所有发送的消息添加标记)
            modified_msg = json.dumps({**msg, "middleware_tag": "processed"})
            data["text"] = modified_msg
        except:
            pass
    await original_send(data)

# 替换recv/send方法, 实现消息拦截
websocket.receive = wrapped_recv
websocket.send = wrapped_send

# 注册WebSocket中间件
app.add_middleware(WebSocketMiddleware, dispatch=custom_ws_middleware)

# 测试实时组件: 输入框 (依赖WebSocket同步)
@ui.page("/")
def index():
    input_box = ui.input("Type something...")
    ui.label("You typed:").bind_text_from(input_box, "value")

ui.run()
```

5. 关键特性

- 握手阶段拦截: 可拒绝非法来源的 WebSocket 连接;
- 消息拦截 / 修改: 支持修改客户端→服务端、服务端→客户端的实时消息;
- 连接状态监听: 可检测连接建立 / 关闭事件, 执行资源清理。

四、两类中间件的核心差异

维度	HTTP 中间件	WebSocket 中间件
协议类型	HTTP/1.1	WebSocket (基于 HTTP 握手, TCP 长连接)
触发时机	每次 HTTP 请求 / 响应	WebSocket 握手 + 连接存续 + 关闭
核心对象	Request / Response	websocket
交互模式	请求 - 响应 (短连接)	双向实时通信 (长连接)
NiceGUI 适用范围	页面加载、API 调用、静态资源	组件实时交互、状态同步、后台推送

五、注意事项

- 1. 中间件执行顺序：先注册的中间件先触发（前置逻辑）、后执行（后置逻辑），需注意链式调用的顺序；
- 2. 性能影响：WebSocket 中间件若逻辑过重（如频繁序列化 / 反序列化），可能影响 NiceGUI 实时交互的流畅性；
- 3. 异常处理：中间件中需捕获异常，避免阻断正常请求 / 连接；
- 4. 与 FastAPI 兼容：NiceGUI 的 `app` 完全兼容 FastAPI 中间件生态，可直接复用 FastAPI 社区的中间件（如跨域、认证、日志中间件）。

详解 NiceGUI 中 `app.add_middleware` 方法

`app.add_middleware` 是 NiceGUI 暴露的核心方法（继承自 Starlette/FastAPI），用于向应用注册中间件，是实现请求 / 连接拦截、自定义处理逻辑的核心入口。以下从方法本质、参数规则、注册逻辑、使用细节、常见场景等维度全面拆解：

一、方法本质与底层关联

NiceGUI 的 `app` 实例（`from nicegui import app`）本质是 Starlette 框架的 `Starlette` 类实例（FastAPI 也基于 Starlette），因此 `app.add_middleware` 完全复用 Starlette 的 `add_middleware` 实现，其核心作用是：

将指定类型的中间件“挂载”到应用的请求 / 连接处理链路中，使所有符合条件的流量（HTTP 请求 / WebSocket 连接）都经过中间件的自定义逻辑。

二、方法核心语法与参数

1. 基础语法

```
from nicegui import app

app.add_middleware(middleware_class, **kwargs)
```

2. 核心参数解析

参数名	类型	必选	说明
<code>middleware_class</code>	中间件类 (Class)	是	指定要注册的中间件类型，核心为两类：① <code>HTTPMiddleware</code> （处理 HTTP）② <code>WebSocketMiddleware</code> （处理 WebSocket）也可传入自定义中间件类（需继承 Starlette 中间件基类）
<code>**kwargs</code>	关键字参数	是	传递给 <code>middleware_class</code> 的初始化参数，最核心的是 <code>dispatch</code> （中间件处理逻辑的回调函数），其他参数可自定义

3. 核心参数: `dispatch` 回调

`dispatch` 是所有中间件的“核心逻辑入口”，不同类型中间件的 `dispatch` 函数签名不同：

中间件类型	<code>dispatch</code> 函数签名（异步推荐）	说明
<code>HTTPMiddleware</code>	<code>async def func(request: Request, call_next) -> Response</code>	<code>request</code> ：HTTP 请求对象； <code>call_next</code> ：传递请求到下一层的回调；返回 <code>Response</code>
<code>WebSocketMiddleware</code>	<code>async def func(websocket: WebSocket, call_next)</code>	<code>websocket</code> ：WebSocket 连接对象； <code>call_next</code> ：传递连接的回调；无返回值（通过操作 <code>websocket</code> 实现逻辑）

三、注册逻辑与执行顺序

1. 注册时机

`app.add_middleware` 需在 `ui.run()` 之前调用（推荐在应用初始化阶段、页面路由定义前），否则中间件可能无法生效。

2. 执行顺序（链式调用）

- 前置逻辑：先注册的中间件 → 后注册的中间件 → NiceGUI 核心处理逻辑；
- 后置逻辑：NiceGUI 核心处理逻辑 → 后注册的中间件 → 先注册的中间件；
- 示例：

```
# 注册顺序：中间件A → 中间件B
app.add_middleware(HTTPMiddleware, dispatch=middleware_a)
app.add_middleware(HTTPMiddleware, dispatch=middleware_b)

# 执行流程：
# 客户端请求 → A前置 → B前置 → NiceGUI处理 → B后置 → A后置 → 响应客户端
```

3. 跨类型中间件的执行关系

HTTP 中间件和 WebSocket 中间件是**独立链路**：

- HTTP 请求仅触发 HTTP 中间件；
- WebSocket 握手请求先触发 HTTP 中间件（因握手基于 HTTP GET），握手成功后建立的 WebSocket 连接仅触发 WebSocket 中间件。

四、关键使用细节

1. 自定义中间件类（进阶）

除了直接使用 `HTTPMiddleware` / `WebSocketMiddleware` + `dispatch` 回调，还可自定义中间件类（继承 `Starlette` 基类），再通过 `app.add_middleware` 注册：

```
from nicegui import app
from starlette.middleware.base import BaseHTTPMiddleware
```

```

from starlette.requests import Request
from starlette.responses import Response

# 自定义HTTP中间件类
class CustomAuthMiddleware(BaseHTTPMiddleware):
    def __init__(self, app, allowed_tokens: list):
        super().__init__(app)
        self.allowed_tokens = allowed_tokens

    async def dispatch(self, request: Request, call_next) -> Response:
        # 自定义鉴权逻辑
        token = request.headers.get("X-Token")
        if token not in self.allowed_tokens:
            return Response("Forbidden", status_code=403)
        response = await call_next(request)
        return response

# 注册自定义中间件类（传入自定义参数）
app.add_middleware(CustomAuthMiddleware, allowed_tokens=["token1", "token2"])

```

2. 异常处理

中间件中需捕获异常，避免阻断整个请求 / 连接链路：

```

async def safe_http_middleware(request: Request, call_next) -> Response:
    try:
        # 前置逻辑
        response = await call_next(request)
        # 后置逻辑
        return response
    except Exception as e:
        # 异常兜底：返回自定义响应
        return Response(f"Error: {str(e)}", status_code=500)

```

3. 与 FastAPI 中间件的兼容

NiceGUI 完全兼容 FastAPI 生态的中间件，可直接复用：

```

# 示例：复用FastAPI的CORS中间件
from nicegui import app
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # 生产环境需限定具体域名
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```


五、常见使用场景

场景	适配中间件类型	实现思路
接口鉴权 / Token 验证	HTTP	前置逻辑校验请求头中的 Token，非法则返回 401/403
请求日志 / 监控	HTTP	记录请求路径、方法、耗时、状态码，输出到日志 / 监控平台
跨域 (CORS) 配置	HTTP	复用 FastAPI 的 <code>CORSMiddleware</code> ，配置允许的源、方法、头
WebSocket 来源验证	WebSocket	前置逻辑校验 <code>origin</code> 头，非法则关闭连接 (code=1008)
实时消息加密 / 解密	WebSocket	重写 <code>websocket.recv/send</code> ，对消息加解密
限流 / 防刷	HTTP/WebSocket	前置逻辑统计请求 / 连接频率，超过阈值则拒绝请求 / 关闭连接

六、注意事项

- 1. 避免重复注册：同一中间件多次注册会导致逻辑重复执行，影响性能；
- 2. 轻量原则：中间件逻辑需简洁（尤其是 WebSocket 中间件），避免阻塞实时交互；
- 3. 作用域：`app.add_middleware` 注册的中间件作用于**整个应用**，若需局部拦截（如单个页面 / API），可使用路由依赖 / 装饰器；
- 4. 版本兼容：Starlette 版本更新可能调整中间件 API，需确保 NiceGUI 与 Starlette 版本匹配（NiceGUI 已内置兼容逻辑）。

装饰器 `@app.add_middleware` 深度解析：简化的中间件注册方式

在 Starlette/NiceGUI 中，`@app.add_middleware` 是对传统“类实例化 + 手动注册”中间件方式的**语法糖封装**，核心目的是简化自定义中间件类的注册流程，让代码更简洁、语义更清晰。以下从「底层原理、使用对比、适用场景、注意事项」四个维度拆解：

一、核心原理：装饰器的本质是“自动注册”

Starlette 的 `app.add_middleware` 本身是一个**方法**（用于注册中间件），但被设计为可作为**类装饰器**使用 —— 当它装饰一个中间件类时，会自动完成「创建类实例 + 注册到应用」的操作，无需手动调用 `app.add_middleware(类名)`。

1. 底层实现逻辑（简化版）

Starlette 对 `add_middleware` 方法的装饰器兼容逻辑可简化管理为：

```

class Starlette:
    def add_middleware(self, middleware_class=None, **kwargs):
        # 情况1: 作为装饰器调用 (无middleware_class参数, 装饰类)
        if middleware_class is None:
            def decorator(cls):
                # 自动创建中间件类实例, 并注册到应用
                self.middleware_stack.append(cls(self))
                return cls
            return decorator
        # 情况2: 手动调用 (传入middleware_class, 如app.add_middleware(HTTPMiddleware, ...))
        else:
            self.middleware_stack.append(middleware_class(self, **kwargs))

```

NiceGUI 完全继承了 Starlette 的这一实现, 因此 `app.add_middleware` 既可以作为**方法手动注册**中间件, 也可以作为**装饰器自动注册**自定义中间件类。

二、两种注册方式对比: 装饰器 vs 手动调用

以你提供的 `AuthMiddleware` 为例, 对比两种写法的差异, 更易理解装饰器的“简化”核心:

1. 装饰器写法 (简化版)

```

from nicegui import app
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import RedirectResponse

# 直接用@app.add_middleware装饰类, 自动注册
@app.add_middleware
class AuthMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        # 鉴权逻辑...
        if not app.storage.user.get('authenticated', False):
            if not request.url.path.startswith('/_nicegui') and request.url.path != '/login':
                return RedirectResponse('/login')
        return await call_next(request)

```

- 核心特点: **类定义 + 注册一步完成**, 无需额外调用 `app.add_middleware(AuthMiddleware)`;
- 语义: 装饰器直接标注“该类是一个要注册的中间件”, 代码可读性更高。

2. 传统手动调用写法 (等价实现)

```

from nicegui import app
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import RedirectResponse

# 第一步: 定义中间件类
class AuthMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        # 相同的鉴权逻辑...

```

```

        if not app.storage.user.get('authenticated', False):
            if not request.url.path.startswith('/_nicegui') and request.url.path !=
'/login':
                return RedirectResponse('/login')
            return await call_next(request)

# 第二步: 手动调用app.add_middleware注册类
app.add_middleware(AuthMiddleware)

```

- 核心特点: **类定义与注册分离**, 需显式调用 `app.add_middleware` 并传入类名;
- 适用场景: 需给中间件类传自定义参数时 (如下文示例)。

三、装饰器写法的限制: 无法传递自定义参数

`@app.add_middleware` 作为装饰器时, **无法直接给中间件类的 `__init__` 传参**—— 因为装饰器调用时没有额外参数入口, 而手动调用则支持。

1. 反例 (装饰器无法传参)

若中间件类需要自定义参数 (如白名单路径), 装饰器写法会报错:

```

# 定义带参数的中间件类
@app.add_middleware # ✗ 装饰器无法传unrestricted_routes参数
class AuthMiddleware(BaseHTTPMiddleware):
    def __init__(self, app, unrestricted_routes: list):
        super().__init__(app)
        self.unrestricted_routes = unrestricted_routes

    async def dispatch(self, request: Request, call_next):
        if request.url.path in self.unrestricted_routes:
            return await call_next(request)
        # 鉴权逻辑...

```

2. 正确写法 (手动调用传参)

此时必须用手动调用方式, 给 `app.add_middleware` 传参:

```

# 定义带参数的中间件类
class AuthMiddleware(BaseHTTPMiddleware):
    def __init__(self, app, unrestricted_routes: list):
        super().__init__(app)
        self.unrestricted_routes = unrestricted_routes

    async def dispatch(self, request: Request, call_next):
        if request.url.path in self.unrestricted_routes:
            return await call_next(request)
        # 鉴权逻辑...

# 手动注册并传递自定义参数
app.add_middleware(AuthMiddleware, unrestricted_routes=['/login', '/register'])

```

四、装饰器写法的适用场景

场景	推荐写法	原因
自定义中间件类（无初始化参数）	装饰器 <code>@app.add_middleware</code>	代码简洁，类定义与注册一体化
自定义中间件类（有初始化参数）	手动调用	需传递参数，装饰器无参数入口
使用内置中间件（如 <code>CORSMiddleware</code> ）	手动调用	内置中间件需传参（如 <code>allow_origins</code> ）

五、关键注意事项

1. 执行时机：无论哪种写法，中间件注册必须在 `ui.run()` / `app.run()` 之前，否则不生效；
2. 继承要求：装饰器装饰的类必须继承 `Starlette` 中间件基类（如 `BaseHTTPMiddleware` / `WebSocketMiddleware`），否则 `Starlette` 无法识别；
3. 兼容性：`@app.add_middleware` 是 `Starlette 2.0+` 支持的语法，`NiceGUI` 已内置兼容，无需担心版本问题；
4. 执行顺序：装饰器注册的中间件，执行顺序与装饰器定义顺序一致（先装饰的先执行前置逻辑）。

六、总结：装饰器写法的核心价值

`@app.add_middleware` 作为装饰器，是 `Starlette/NiceGUI` 为“无参数自定义中间件”提供的语法糖：

- 简化代码：省去手动调用 `app.add_middleware` 的步骤；
- 语义清晰：装饰器直接标注类的用途（“这是一个要注册的中间件”）；
- 限制明确：仅适用于无初始化参数的中间件类，有参数时需回归手动调用。

深度解析 `BaseHTTPMiddleware`
(`Starlette/NiceGUI` 核心 HTTP 中间件基类)

`BaseHTTPMiddleware` 是 `Starlette` 框架提供的**标准 HTTP 中间件基类**，也是 `NiceGUI` 中自定义 HTTP 中间件的核心依赖。它封装了 HTTP 请求 / 响应的处理流程，提供标准化的扩展接口，让开发者无需关注底层协议细节，只需实现核心业务逻辑。以下从「核心定位、设计原理、使用规范、高级特性、对比其他中间件类」等维度全面拆解：

一、核心定位：为什么选择 `BaseHTTPMiddleware`

在 `Starlette` 生态中，处理 HTTP 中间件有两类核心方式：

1. 基础的 `HTTPMiddleware`（简单 `dispatch` 回调）；
2. 可扩展的 `BaseHTTPMiddleware`（类继承式扩展）。

`BaseHTTPMiddleware` 的核心优势是：

- **面向对象扩展**：支持通过类继承封装状态（如自定义参数、缓存）；
- **完整生命周期**：显式处理请求前置、响应后置逻辑；
- **兼容性强**：`NiceGUI/FastAPI` 均原生支持，是自定义复杂 HTTP 中间件的首选。

二、底层设计原理

`BaseHTTPMiddleware` 基于 Starlette 的 `Middleware` 抽象层实现，核心逻辑可简化为：

```
class BaseHTTPMiddleware:
    def __init__(self, app):
        self.app = app # 存储下一层应用/中间件的引用（链式调用核心）

    async def dispatch(self, request: Request, call_next):
        """核心扩展接口：开发者需重写该方法实现自定义逻辑"""
        # 前置逻辑（默认空）
        response = await call_next(request) # 传递请求到下一层
        # 后置逻辑（默认空）
        return response

    async def __call__(self, scope, receive, send):
        """Starlette 中间件核心入口（无需开发者重写）"""
        # 仅处理 HTTP 类型请求（过滤 WebSocket/ASGI 其他类型）
        if scope["type"] != "http":
            await self.app(scope, receive, send)
            return
        # 封装 scope/receive/send 为 Request 对象，调用 dispatch
        request = Request(scope, receive=receive)
        call_next = lambda: self.app(scope, receive, send)
        response = await self.dispatch(request, call_next)
        # 将 Response 对象转换为 ASGI 协议的 send 调用
        await response(scope, receive, send)
```

关键逻辑：

- `__call__` 方法：是 ASGI 协议的核心入口（Starlette 自动调用），负责过滤 HTTP 请求、封装 `Request` 对象，开发者无需修改；
- `dispatch` 方法：是留给开发者的**扩展接口**，只需重写该方法即可实现自定义拦截逻辑；
- 链式调用：`self.app` 指向“下一层中间件 / 应用核心逻辑”，`call_next` 实现请求的传递。

三、核心使用规范（必掌握）

1. 基础使用步骤

```
from nicegui import app
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import Response

# 步骤1: 继承 BaseHTTPMiddleware
class CustomMiddleware(BaseHTTPMiddleware):
    # 步骤2（可选）：自定义初始化参数
    def __init__(self, app, custom_param: str):
        super().__init__(app) # 必须调用父类__init__，传入app
        self.custom_param = custom_param # 封装自定义状态

# 步骤3: 重写 dispatch 方法（核心逻辑）
```

```
async def dispatch(self, request: Request, call_next) -> Response:
    # 阶段1: 请求前置逻辑 (拦截请求、鉴权、日志等)
    print(f"请求路径: {request.url.path}, 自定义参数: {self.custom_param}")

    # 阶段2: 传递请求到下一层 (核心! 否则请求会被阻断)
    response = await call_next(request)

    # 阶段3: 响应后置逻辑 (修改响应、统计耗时等)
    response.headers["X-Custom-Header"] = self.custom_param

    # 阶段4: 返回响应 (必须返回 Response 对象)
    return response

# 步骤4: 注册中间件 (NiceGUI 中)
app.add_middleware(CustomMiddleware, custom_param="hello-nicegui")
```

2. `dispatch` 方法核心参数

参数名	类型	作用
<code>request</code>	<code>starlette.requests.Request</code>	封装 HTTP 请求的所有信息: 路径 (<code>url.path</code>)、方法 (<code>method</code>)、头 (<code>headers</code>)、参数 (<code>query_params</code>) 等
<code>call_next</code>	异步回调函数	调用 <code>await call_next(request)</code> 会将请求传递到下一层 (中间件 / 应用逻辑), 返回 <code>Response</code> 对象
返回值	<code>starlette.responses.Response</code>	必须返回 <code>Response</code> 子类 (如 <code>RedirectResponse</code> / <code>JSONResponse</code>), 否则会抛出异常

3. 关键约束

- 必须异步: `dispatch` 方法必须定义为 `async` (Starlette 强制要求, 否则无法兼容异步事件循环);
- 父类初始化: 自定义 `__init__` 时必须调用 `super().__init__(app)`, 否则 `self.app` 为空, 导致 `call_next` 失效;
- 仅处理 HTTP: `BaseHTTPMiddleware` 只拦截 `scope["type"] == "http"` 的请求, WebSocket 连接不会触发。

四、高级特性与实战场景

1. 拦截并修改响应体

`BaseHTTPMiddleware` 支持读取 / 修改响应体 (需注意响应体是字节流):

```
class ModifyBodyMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next) -> Response:
        response = await call_next(request)
        # 读取原始响应体
        original_body = await response.body()
        # 修改响应体 (示例: 添加自定义文本)
        new_body = original_body + b" [Modified by Middleware]"
        # 重新构建响应 (需保留原状态码/头)
        return Response(
            content=new_body,
```

```
        status_code=response.status_code,
        headers=response.headers,
        media_type=response.media_type
    )
```

2. 异常捕获与兜底

在 `dispatch` 中捕获请求处理过程中的异常，返回自定义兜底响应：

```
class ErrorHandlerMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next) -> Response:
        try:
            return await call_next(request)
        except Exception as e:
            # 捕获所有异常，返回500响应
            return Response(
                content=f"Server Error: {str(e)}",
                status_code=500,
                media_type="text/plain"
            )
```

3. 结合 NiceGUI 存储实现用户态拦截

利用 `app.storage.user` 实现用户级的动态拦截（如你之前的鉴权场景）：

```
class AuthMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next) -> Response:
        # 读取用户认证状态
        is_authed = app.storage.user.get("authenticated", False)
        # 白名单路径放行
        if request.url.path in ["/login", "/_nicegui"]:
            return await call_next(request)
        # 未认证则重定向
        if not is_authed:
            return RedirectResponse(f"/login?redirect={request.url.path}")
        return await call_next(request)
```

五、BaseHTTPMiddleware vs HTTPMiddleware（核心对比）

Starlette 还提供了基础的 `HTTPMiddleware`，两者的核心差异决定了使用场景：

维度	BaseHTTPMiddleware	HTTPMiddleware
扩展方式	类继承（重写 <code>dispatch</code> ）	函数回调（传入 <code>dispatch</code> 参数）
状态封装	支持（通过类属性存储）	不支持（需用闭包 / 全局变量）
自定义参数	支持（通过 <code>__init__</code> 传参）	支持（通过 <code>**kwargs</code> 传参）
代码结构	面向对象，适合复杂逻辑	函数式，适合简单逻辑
NiceGUI 推荐度	高（自定义中间件首选）	低（仅临时简单拦截使用）

示例：HTTPMiddleware 的简单用法（对比参考）

```
from starlette.middleware.base import HTTPMiddleware

# 函数式 dispatch 回调
async def simple_dispatch(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Simple"] = "true"
    return response

# 注册
app.add_middleware(HTTPMiddleware, dispatch=simple_dispatch)
```

六、常见坑点与避坑指南

1. **忘记调用 `call_next`**：会导致请求被阻断，客户端一直等待响应；
2. **同步代码阻塞**：`dispatch` 中避免执行耗时同步操作（如文件读写），需用 `asyncio.to_thread` 包装；
3. **响应体重重复读取**：`response.body()` 是可耗尽的，多次读取需先缓存；
4. **注册时机错误**：必须在 `ui.run()` 前注册，否则中间件不生效；
5. **路径匹配错误**：注意 `request.url.path` 是绝对路径（如 `/login`），避免漏写斜杠。

七、总结

`BaseHTTPMiddleware` 是 Starlette/NiceGUI 中自定义 HTTP 中间件的标准基类，其核心价值是：

- 封装 ASGI 底层细节，提供面向开发者的 `dispatch` 扩展接口；
- 支持面向对象的扩展方式，便于封装状态和自定义参数；
- 完整覆盖 HTTP 请求的“前置 - 传递 - 后置”生命周期，满足鉴权、日志、限流等几乎所有 HTTP 拦截场景。

在 NiceGUI 开发中，凡是需要全局拦截 HTTP 请求（页面访问、API 调用、静态资源加载）的场景，优先选择继承 `BaseHTTPMiddleware` 实现自定义中间件。

案例代码

```
@app.add_middleware
class AuthMiddleware(BaseHTTPMiddleware):
    """This middleware restricts access to all NiceGUI pages.

    It redirects the user to the login page if they are not authenticated.
    """

    async def dispatch(self, request: Request, call_next):
        if not app.storage.user.get('authenticated', False):
            if not request.url.path.startswith('/_nicegui') and request.url.path not in
unrestricted_page_routes:
                return RedirectResponse(f'/login?redirect_to={request.url.path}')
            return await call_next(request)
```


这段代码是 NiceGUI 中典型的**全局 HTTP 鉴权中间件**，核心作用是拦截所有 HTTP 请求，校验用户是否已认证，未认证则重定向到登录页（仅放行白名单路径）。以下分模块拆解：

一、整体结构与核心依赖

```
@app.add_middleware
class AuthMiddleware(BaseHTTPMiddleware):
    """This middleware restricts access to all NiceGUI pages.

    It redirects the user to the login page if they are not authenticated.
    """

    async def dispatch(self, request: Request, call_next):
        if not app.storage.user.get('authenticated', False):
            if not request.url.path.startswith('/_nicegui') and request.url.path not in
unrestricted_page_routes:
                return RedirectResponse(f'/login?redirect_to={request.url.path}')
            return await call_next(request)
```

- 核心继承： `BaseHTTPMiddleware` 是 Starlette 提供的 HTTP 中间件基类，所有自定义 HTTP 中间件需继承它；
- 装饰器用法： `@app.add_middleware` 是 NiceGUI/Starlette 简化的中间件注册方式（替代 `app.add_middleware(AuthMiddleware)`），直接将类注册为全局中间件；
- `dispatch` 方法：中间件的核心逻辑入口，接收 `request`（当前 HTTP 请求对象）和 `call_next`（传递请求到下一层的回调），返回 `Response`（或子类如 `RedirectResponse`）。

二、逐行逻辑拆解

1. 类定义与注释

```
class AuthMiddleware(BaseHTTPMiddleware):
    """This middleware restricts access to all NiceGUI pages.

    It redirects the user to the login page if they are not authenticated.
    """
```

- 功能声明：该中间件用于限制所有 NiceGUI 页面的访问权限，未认证用户会被重定向到登录页；
- 基类选择： `BaseHTTPMiddleware` 是 Starlette 推荐的 HTTP 中间件基类，相比基础的 `HTTPMiddleware` 更易扩展（支持自定义初始化参数）。

2. 核心 `dispatch` 方法

```
async def dispatch(self, request: Request, call_next):
```

- 异步声明： `async` 表示该方法是异步的（Starlette 推荐异步中间件，避免阻塞事件循环）；
- 参数说明：
 - `request: Request`：封装了当前 HTTP 请求的所有信息（路径、头、参数、客户端信息等）；
 - `call_next`：回调函数，调用 `await call_next(request)` 会将请求传递到下一层（如 NiceGUI 的

路由处理逻辑、其他中间件)，返回最终的响应对象。

3. 认证状态校验

```
if not app.storage.user.get('authenticated', False):
```

- `app.storage.user`：NiceGUI 内置的**用户级存储**（基于 Cookie/Session），用于存储当前用户的状态（如认证标识、用户信息），不同用户的 `storage.user` 相互隔离；
- `get('authenticated', False)`：读取 `authenticated` 字段，若不存在则默认返回 `False`；
- 逻辑含义：如果用户未认证（`authenticated` 为 `False`），进入后续的重定向判断逻辑；若已认证，直接放行请求。

4. 白名单路径放行

```
if not request.url.path.startswith('/_nicegui') and request.url.path not in  
unrestricted_page_routes:
```

这是**关键的白名单逻辑**，避免中间件拦截必要的资源或公开页面：

- `request.url.path`：当前请求的路径（如 `/`、`/login`、`/_nicegui/js/app.js`）；
- `not request.url.path.startswith('/_nicegui')`：
 - `/_nicegui` 是 NiceGUI 内置的静态资源路径（包含 JS/CSS/ 组件依赖），若拦截该路径会导致页面样式、交互失效；
 - 逻辑含义：**如果请求路径不是 `/_nicegui` 开头**（即不是内置资源），才继续判断；
- `request.url.path not in unrestricted_page_routes`：
 - `unrestricted_page_routes` 是开发者定义的“公开页面路径列表”（如 `['/login', '/register']`）；
 - 逻辑含义：**如果请求路径不在公开列表中**，才触发重定向。

5. 重定向到登录页

```
return RedirectResponse(f'/login?redirect_to={request.url.path}')
```

- `RedirectResponse`：Starlette 提供的重定向响应类，返回 307/308 状态码，引导客户端跳转到指定路径；
- `f'/login?redirect_to={request.url.path}'`：
 - 跳转到 `/login` 页面；
 - 携带 `redirect_to` 参数，记录用户原本要访问的路径（登录成功后可跳转回该路径）；
- 逻辑含义：未认证用户访问非白名单路径时，强制跳转到登录页。

6. 放行请求

```
return await call_next(request)
```

- 若用户已认证，或请求路径在白名单中，调用 `call_next(request)` 将请求传递到下一层（NiceGUI 的路由处理逻辑），并返回最终的响应；

- 这是中间件“放行”的核心操作，确保合法请求能正常访问目标页面。

三、核心功能总结

场景	中间件行为
已认证用户	放行所有请求（包括页面、API、静态资源）
未认证用户访问 <code>/login</code>	放行（因 <code>/login</code> 在 <code>unrestricted_page_routes</code> 中）
未认证用户访问 <code>/</code>	重定向到 <code>/login?redirect_to=/'</code>
未认证用户访问 <code>/_nicegui/js/app.js</code>	放行（内置资源路径）

四、关键补充说明

1. `unrestricted_page_routes` 的定义：

代码中未显示该变量，需开发者提前定义，示例：

```
unrestricted_page_routes = ['/login', '/register', '/forgot-password']
```

2. 认证状态的设置：

登录页需在用户验证通过后，设置 `app.storage.user['authenticated'] = True`，示例：

```
@ui.page('/login')
def login_page():
    async def handle_login():
        if username.value == 'admin' and password.value == '123456':
            app.storage.user['authenticated'] = True # 标记为已认证
            ui.navigate.to(request.query_params.get('redirect_to', '/')) # 跳转回原路径
        else:
            ui.notify('用户名或密码错误')

    username = ui.input('用户名')
    password = ui.input('密码').password()
    ui.button('登录', on_click=handle_login)
```

3. 中间件生效范围：

该中间件是全局生效的，会拦截所有 HTTP 请求（包括页面访问、API 调用、静态资源加载），但通过白名单避免了必要资源的拦截。

五、潜在优化点

- 排除 API 路径：若有公开 API（如 `/api/public`），需加入白名单；
- 避免重复重定向：可判断 `request.url.path` 是否为 `/login`，防止循环重定向；
- 支持排除 HTTP 方法：如放行 OPTIONS 请求（跨域预检）。